

Oshudh Somadhan

Go Backend Learning Roadmap & Project Plan

From medicine catalog to production-grade microservices

Project focus	Description
Domain	Bangladesh medicine catalog, pharmacy inventory, prescription order, payment, and notification platform
Primary learning goal	Learn Go deeply by building a real backend system step by step
Initial database base	medicine, generics, companie, dosageform
Final architecture	API Gateway, gRPC microservices, PostgreSQL, Redis, RabbitMQ/NATS, GraphQL, observability, testing, Docker/Kubernetes

Important note: This project is for learning software engineering and medicine-data management. Do not present medicine suggestions as professional medical advice. Always add disclaimers and require users to consult doctors/pharmacists.

Document Map

- Part 1: Project vision and final product scope
- Part 2: Recommended architecture
- Part 3: Phase-by-phase learning roadmap from easy to complex
- Part 4: Database expansion plan
- Part 5: Service communication and event flow
- Part 6: Testing, DevOps, deployment, and monitoring plan
- Part 7: Suggested GitHub milestone structure

Part 1: Project Vision

Oshudh Somadhan will be a Bangladesh-focused medicine information and pharmacy platform. The first version should start as a simple medicine search system using your current data. Later it should grow into a real-world backend system with users, roles, pharmacies, inventory, cart, orders, prescriptions, payments, notifications, caching, GraphQL, gRPC, tests, and deployment.

Core user types

User type	What they can do
Customer	Search medicines, compare alternatives, upload prescription, add to cart, place orders, pay, track order status
Admin	Manage medicines, companies, generics, users, pharmacies, orders, payments, and reports
Pharmacy owner/staff	Manage pharmacy branches, stock, batches, expiry, order fulfillment
Delivery user	View assigned deliveries, update delivery status
System worker	Process events, send notifications, update cache, run scheduled jobs

Final product modules

Module	Main responsibility
Medicine catalog	Search, filter, details, generic-based alternatives, company/dosage form browsing
Auth & RBAC	Login, registration, refresh token, permissions, admin/customer/pharmacy access
Inventory	Pharmacy-wise stock, batches, expiry, stock movements, low-stock alert
Cart & Order	Cart, checkout, order state machine, prescription order flow
Payment	Payment records, provider callback/webhook, refund state, invoice
Prescription	Upload, review, approve/reject, attach medicines to prescription
Notification	Email/SMS/push/in-app notifications using events
Recommendation	Alternative medicines, cheaper alternatives, popular searches
Admin analytics	Orders, payment, stock, medicine search trends, reports

Part 2: Recommended Architecture

Start with a modular monolith, then split into microservices. This avoids early complexity and lets you learn the domain first.

```

Client Web/Mobile/Admin
    |
    v
API Gateway (REST + GraphQL)
    |
    | gRPC calls
    v
Auth Service | Medicine Service | Inventory Service | Order Service
Payment Service | Prescription Service | Notification Service | Recommendation Service
    |
    v
PostgreSQL + Redis + RabbitMQ/NATS + File Storage
    |
    v
Prometheus + Grafana + OpenTelemetry + Jaeger

```

Technology	Recommended choice	Why
HTTP framework	Gin or Chi	Simple, popular, production-friendly
Database	PostgreSQL	Strong relational modeling, full-text search, transactions
DB access	sqlc first, GORM later optional	sqlc teaches SQL deeply and gives type-safe Go code
Migration	golang-migrate or goose	Version-controlled schema changes
Cache	Redis	Search/detail cache, sessions, rate limit, OTP
gRPC	grpc-go + protobuf	Internal service communication
GraphQL	gqlgen	Frontend-friendly medicine/order queries
Queue/events	RabbitMQ first, NATS later optional	Order/payment/notification async workflow
Testing	testing + testify + gomock + testcontainers	Unit, integration, DB, and service tests
Observability	Zap/ZeroLog + Prometheus + OpenTelemetry	Production-grade monitoring and tracing

Part 3: Phase-by-Phase Learning Roadmap

Phase	Build focus	Main topic learned
0	Setup, repository, Docker, PostgreSQL import	Go project setup, tooling, migrations
1	Medicine catalog API	REST, SQL, repositories, pagination
2	Clean architecture refactor	Layering, interfaces, dependency injection
3	Auth and RBAC	JWT, refresh token, password hashing, permissions
4	Search and cache	PostgreSQL full-text search, Redis, query optimization
5	Admin CRUD and audit logs	Validation, admin APIs, audit trails
6	Pharmacy inventory	Transactions, stock movement, row locking
7	Cart and order system	Order lifecycle, consistency, state machine
8	Prescription workflow	File upload, review workflow, storage
9	Payment module	Webhook, idempotency, payment states
10	Notification and event bus	RabbitMQ/NATS, async workers, retries
11	gRPC internal services	Protobuf, service contracts, interceptors
12	GraphQL API layer	gqlgen, query aggregation, frontend-friendly API
13	Testing depth	Unit, integration, mocks, testcontainers

14	Observability	Logging, metrics, tracing, health checks
15	DevOps and deployment	Docker Compose, CI/CD, Kubernetes

Phase 0: Foundation Setup

Item	Details
Difficulty	Beginner
Suggested duration	2-4 days
Main goal	Create the base repository and local development environment for Oshudh Somadhan.

What you will build

- Create GitHub repository and README.
- Set up Go module.
- Set up PostgreSQL using Docker Compose.
- Import existing backup.sql.
- Add environment configuration.
- Add basic Makefile commands.

What you will learn in Go/backend

- Go module structure.
- Environment variables.
- Docker Compose basics.
- PostgreSQL import/export.
- Migrations and local developer workflow.

Database / schema work

- Keep your existing tables: medicine, generics, companie, dosageform.
- Create a migrations folder even if the initial schema comes from backup.sql.
- Plan to rename companie to companies in a later migration.

API / service work

- No complex API yet. Add only a health endpoint: GET /health.
- Add DB ping endpoint only for development: GET /debug/db-ping.

Testing focus

- Test config loading.
- Test DB connection function.
- Test health endpoint using httptest.

Exit criteria before moving to next phase

- Project runs with one command.
- PostgreSQL container starts successfully.
- backup.sql imported successfully.

- GET /health returns OK.

Phase 1: Medicine Catalog API

Item	Details
Difficulty	Beginner
Suggested duration	1 week
Main goal	Build the first useful API from your existing medicine database.

What you will build

- List medicines with pagination.
- Medicine detail by ID.
- Search by medicine name.
- Filter by generic, company, dosage form.
- Show alternatives by same generic.
- Show generic detail and its medicines.

What you will learn in Go/backend

- HTTP handlers in Go.
- Repository pattern basics.
- SQL joins.
- Pagination and query params.
- DTO/response design.
- Error handling.

Database / schema work

- Add indexes for medicine.name, medicine.generics_id, medicine.companie_id, medicine.dosageform_id.
- Consider adding created_at and updated_at later.
- Create a view for medicine detail if needed.

API / service work

- GET /api/v1/medicines
- GET /api/v1/medicines/{id}
- GET /api/v1/medicines/search?q=napa
- GET /api/v1/generics/{id}/medicines
- GET /api/v1/medicines/{id}/alternatives

Testing focus

- Repository tests for medicine query.
- Handler tests for success and not found.
- Pagination tests.
- Invalid query param tests.

Exit criteria before moving to next phase

- You can search medicine by name.
- You can see medicine detail with generic/company/dosage form.
- You can see alternatives from the same generic.

Phase 2: Clean Architecture Refactor

Item	Details
Difficulty	Beginner to Intermediate
Suggested duration	4-6 days
Main goal	Refactor the first working API into a maintainable Go project structure.

What you will build

- Separate handler, service/usecase, repository, domain model, DTO.
- Add centralized error response.
- Add request validation.
- Add config package.
- Add logger package.

What you will learn in Go/backend

- Clean architecture.
- Interfaces in Go.
- Dependency injection without magic.
- Package boundaries.
- Context propagation.
- Structured error handling.

Database / schema work

- No major schema change.
- Add migration tracking table through migration tool.
- Create seed scripts for local development.

API / service work

- Keep same APIs but move business logic out of handlers.
- Add consistent response format.
- Add request ID middleware.

Testing focus

- Unit test service layer using mocked repository.
- Handler test using fake service.
- Repository integration test against real PostgreSQL.

Exit criteria before moving to next phase

- Handlers are thin.

- Business logic is testable without HTTP.
- Repository only handles database work.
- Codebase is easy to extend.

Phase 3: Authentication and RBAC

Item	Details
Difficulty	Intermediate
Suggested duration	1-2 weeks
Main goal	Add real users, login, refresh token, roles, and permissions.

What you will build

- Customer registration and login.
- Admin login.
- Password hashing.
- JWT access token.
- Refresh token rotation.
- Role-based permission checks.
- Protected admin routes.

What you will learn in Go/backend

- JWT architecture.
- Refresh token security.
- bcrypt/argon2 password hashing.
- Middleware.
- RBAC modeling.
- Security basics.

Database / schema work

- Create users, roles, permissions, user_roles, role_permissions, refresh_tokens, password_resets.
- Add unique constraints on email/phone.
- Store hashed passwords only.

API / service work

- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout
- GET /api/v1/me
- Admin-only middleware for /admin routes.

Testing focus

- Password hash verification test.
- JWT generation/validation test.

- Middleware unauthorized/forbidden test.
- Refresh token reuse prevention test.

Exit criteria before moving to next phase

- Users can register/login.
- Admin route blocks normal users.
- Refresh token works securely.
- You understand auth flow clearly.

Phase 4: Advanced Search, Redis Cache, and Performance

Item	Details
Difficulty	Intermediate
Suggested duration	1 week
Main goal	Make medicine search faster and more production-like.

What you will build

- PostgreSQL full-text search.
- Autocomplete endpoint.
- Popular search tracking.
- Redis cache for medicine detail and search result.
- Cache invalidation on admin update.
- Rate limiting.

What you will learn in Go/backend

- PostgreSQL full-text search.
- GIN index basics.
- Redis data structures.
- Cache-aside pattern.
- TTL strategy.
- Rate limiting middleware.
- Performance measurement.

Database / schema work

- Add medicine_search_logs.
- Add search_vector generated column or separate search table.
- Add GIN index for search.
- Add medicine_views table optional.

API / service work

- GET /api/v1/search/medicines?q=
- GET /api/v1/search/suggestions?q=
- GET /api/v1/medicines/popular
- Cache GET /medicines/{id}.

Testing focus

- Test cache miss then cache hit.
- Test cache invalidation after update.
- Benchmark search query before/after index.
- Rate limit test.

Exit criteria before moving to next phase

- Search is fast for common queries.
- Redis is used intentionally, not randomly.
- You can explain cache hit, miss, TTL, and invalidation.

Phase 5: Admin CRUD, Validation, and Audit Logs

Item	Details
Difficulty	Intermediate
Suggested duration	1 week
Main goal	Build admin APIs to manage the catalog safely.

What you will build

- CRUD for medicines.
- CRUD for generics.
- CRUD for companies.
- CRUD for dosage forms.
- Validation and duplicate prevention.
- Audit log for admin actions.

What you will learn in Go/backend

- Admin API design.
- Input validation.
- Database constraints.
- Audit logging.
- Soft delete vs hard delete.
- Transactional update.

Database / schema work

- Rename companie to companies through migration if you want clean naming.
- Add audit_logs table.
- Add deleted_at optional for soft delete.
- Add updated_by user reference optional.

API / service work

- POST /admin/medicines
- PUT /admin/medicines/{id}
- DELETE /admin/medicines/{id}

- GET /admin/audit-logs

Testing focus

- Validation tests.
- Admin permission tests.
- Audit log creation test.
- Duplicate medicine handling test.

Exit criteria before moving to next phase

- Admin can manage catalog.
- Every sensitive change is logged.
- Invalid data is rejected before reaching DB.

Phase 6: Pharmacy and Inventory Service

Item	Details
Difficulty	Intermediate to Advanced
Suggested duration	1-2 weeks
Main goal	Add pharmacy-wise stock management to make the platform realistic.

What you will build

- Create pharmacies and branches.
- Add medicine stock per branch.
- Track batch number and expiry date.
- Stock in/out movement.
- Low stock alert.
- Expiry warning list.

What you will learn in Go/backend

- Inventory modeling.
- PostgreSQL transactions.
- Concurrency issues.
- SELECT FOR UPDATE.
- Stock movement ledger.
- Consistency rules.

Database / schema work

- Create pharmacies, pharmacy_branches, medicine_stocks, stock_batches, stock_movements.
- Use stock_movements as source of truth for stock history.
- Add indexes by pharmacy, medicine, expiry date.

API / service work

- POST /admin/pharmacies
- POST /pharmacy/stocks

- POST /pharmacy/stocks/adjust
- GET /pharmacy/stocks/low
- GET /medicines/{id}/availability

Testing focus

- Stock increase/decrease transaction test.
- Prevent negative stock test.
- Concurrent order stock reservation test.
- Expiry date filtering test.

Exit criteria before moving to next phase

- Stock cannot become negative.
- Every stock change creates a movement record.
- Availability API shows pharmacy-wise stock.

Phase 7: Cart and Order System

Item	Details
Difficulty	Advanced
Suggested duration	1-2 weeks
Main goal	Build the core business workflow: cart, checkout, and order state management.

What you will build

- Customer cart.
- Add/remove/update cart items.
- Checkout.
- Create order.
- Order items snapshot.
- Order status history.
- Cancel order rules.

What you will learn in Go/backend

- Order state machine.
- Transactional order creation.
- Idempotency.
- Data snapshot strategy.
- Business rules.
- Service-level consistency.

Database / schema work

- Create carts, cart_items, orders, order_items, order_status_logs.
- Store medicine name/price snapshot in order_items.
- Add idempotency_keys table optional.

API / service work

- POST /cart/items
- PATCH /cart/items/{id}
- DELETE /cart/items/{id}
- POST /orders/checkout
- GET /orders/{id}
- PATCH /admin/orders/{id}/status

Testing focus

- Checkout transaction test.
- Order item snapshot test.
- Cancel order rule test.
- Idempotent checkout test.
- Unauthorized order access test.

Exit criteria before moving to next phase

- Customer can place an order.
- Order has clear status history.
- Stock reservation behavior is defined.
- Repeated checkout request does not create duplicate order.

Phase 8: Prescription Workflow

Item	Details
Difficulty	Advanced
Suggested duration	1 week
Main goal	Support prescription-required orders and file uploads.

What you will build

- Upload prescription image/PDF.
- Review prescription by pharmacist/admin.
- Approve/reject prescription.
- Attach prescription to order.
- Prescription item extraction manually first.

What you will learn in Go/backend

- Multipart file upload.
- File validation.
- Local storage vs object storage.
- Review workflow.
- Secure file access.
- Background processing later.

Database / schema work

- Create prescriptions, prescription_items, prescription_reviews.
- Store file path or object key, not raw files in DB.
- Add status: PENDING, APPROVED, REJECTED.

API / service work

- POST /prescriptions
- GET /prescriptions/{id}
- PATCH /admin/prescriptions/{id}/review
- POST /orders/checkout-with-prescription

Testing focus

- File type validation test.
- Max file size test.
- Review permission test.
- Prescription status transition test.

Exit criteria before moving to next phase

- Prescription can be uploaded securely.
- Admin can review it.
- Rejected prescription cannot create confirmed order.

Phase 9: Payment Service

Item	Details
Difficulty	Advanced
Suggested duration	1-2 weeks
Main goal	Add payment records, simulated gateway flow, webhook/callback handling, and invoices.

What you will build

- Cash on delivery.
- Online payment simulation.
- Payment initiation.
- Payment callback/webhook.
- Payment status update.
- Refund request.
- Invoice record.

What you will learn in Go/backend

- Payment state machine.
- Webhook security.
- Idempotency.
- Provider abstraction.

- Money handling.
- Transaction logs.

Database / schema work

- Create payments, payment_transactions, refunds, invoices.
- Use numeric/decimal for money.
- Add unique transaction reference.
- Add payment status logs optional.

API / service work

- POST /payments/initiate
- POST /payments/webhook/mock
- GET /payments/{id}
- POST /admin/refunds
- GET /orders/{id}/invoice

Testing focus

- Webhook idempotency test.
- Payment status transition test.
- Invalid signature test later.
- Refund rule test.
- Money rounding test.

Exit criteria before moving to next phase

- Order can move to paid after valid payment callback.
- Duplicate webhook does not duplicate payment.
- Payment history is auditable.

Phase 10: Notification and Event Bus

Item	Details
Difficulty	Advanced
Suggested duration	1 week
Main goal	Decouple order/payment/prescription events from notification delivery.

What you will build

- Publish events when important actions happen.
- Notification worker consumes events.
- Send email/SMS/push mock.
- In-app notification list.
- Retry failed notification.
- Dead letter queue concept.

What you will learn in Go/backend

- Event-driven architecture.
- RabbitMQ/NATS basics.
- Producer/consumer pattern.
- Retry strategy.
- Dead letter queue.
- Outbox pattern introduction.

Database / schema work

- Create notifications, notification_templates, notification_logs, outbox_events optional.
- Use event_id to avoid duplicate processing.

API / service work

- Worker consumes OrderPlaced, PaymentCompleted, PrescriptionReviewed.
- GET /notifications
- PATCH /notifications/{id}/read

Testing focus

- Consumer handler test.
- Duplicate event handling test.
- Retry simulation test.
- Notification template rendering test.

Exit criteria before moving to next phase

- Order flow no longer directly sends messages.
- Events can be replayed safely.
- Notification worker can fail without breaking checkout.

Phase 11: gRPC Microservice Communication

Item	Details
Difficulty	Advanced
Suggested duration	1-2 weeks
Main goal	Start splitting the modular monolith into real internal services using gRPC.

What you will build

- Create proto files.
- MedicineService gRPC.
- AuthService gRPC token validation.
- InventoryService reserve/release stock.
- OrderService calls InventoryService internally.
- gRPC interceptors for logging/auth.

What you will learn in Go/backend

- Protocol Buffers.
- grpc-go.
- Service contracts.
- Unary interceptors.
- gRPC error codes.
- Deadline and context cancellation.
- Backward-compatible API evolution.

Database / schema work

- Each service can still use same PostgreSQL during learning, but design boundaries carefully.
- Later each service can own its schema or database.

API / service work

- AuthService.ValidateToken
- MedicineService.GetMedicine
- InventoryService.ReserveStock
- OrderService.CreateOrder

Testing focus

- gRPC handler test.
- Client timeout test.
- Interceptor test.
- Reserve stock integration test.

Exit criteria before moving to next phase

- At least Auth, Medicine, Inventory, and Order communicate through gRPC.
- Proto contracts are version-controlled.
- You understand REST vs gRPC responsibility.

Phase 12: GraphQL API Layer

Item	Details
Difficulty	Advanced
Suggested duration	1 week
Main goal	Add GraphQL for flexible frontend queries while keeping services clean.

What you will build

- GraphQL endpoint at API Gateway.
- Medicine detail query.
- Alternatives query.
- Order history query.
- Admin dashboard summary query.
- Avoid N+1 problem using dataloader pattern.

What you will learn in Go/backend

- GraphQL schema design.
- Resolvers.
- Query vs mutation design.
- DataLoader.
- Aggregation across services.
- Frontend-friendly APIs.

Database / schema work

- No major schema change.
- Add query logs optional for analytics.

API / service work

- query medicine(id)
- query searchMedicines
- query me
- query myOrders
- query adminDashboardSummary

Testing focus

- Resolver test.
- Unauthorized GraphQL query test.
- N+1 prevention test with query count.
- Schema validation test.

Exit criteria before moving to next phase

- Frontend can fetch nested medicine details in one request.
- GraphQL does not bypass auth.
- Resolvers stay thin and use service layer.

Phase 13: Deep Testing Strategy

Item	Details
Difficulty	Advanced
Suggested duration	1-2 weeks
Main goal	Make the project reliable and professional with multiple testing layers.

What you will build

- Unit tests for service logic.
- Repository integration tests.
- HTTP handler tests.
- gRPC tests.
- Payment webhook tests.
- Order transaction tests.

- Cache tests.
- Contract tests optional.

What you will learn in Go/backend

- Go testing package deeply.
- Testify.
- Mocks using gomock or mockery.
- Testcontainers.
- Table-driven tests.
- Race testing.
- Benchmarking.

Database / schema work

- Use isolated test DB.
- Run migrations before tests.
- Seed minimal data for each test.
- Avoid dependency on your full production dataset.

API / service work

- No new API required. Add test coverage around all critical APIs.

Testing focus

- Run go test ./...
- Run integration tests with Docker.
- Run go test -race.
- Add benchmark for search endpoint.

Exit criteria before moving to next phase

- Critical business rules have tests.
- CI runs tests automatically.
- You can refactor without fear.

Phase 14: Observability and Reliability

Item	Details
Difficulty	Advanced
Suggested duration	1 week
Main goal	Add production-style logging, metrics, tracing, and health checks.

What you will build

- Structured logs.
- Request ID.
- Prometheus metrics.
- OpenTelemetry tracing.

- Jaeger tracing.
- Health/readiness checks.
- Graceful shutdown.

What you will learn in Go/backend

- Observability basics.
- Metrics vs logs vs traces.
- Context propagation.
- Graceful shutdown.
- Timeout and retry rules.
- Circuit breaker concept.

Database / schema work

- No required schema change.
- Add service_health_logs optional only if needed.

API / service work

- GET /health/live
- GET /health/ready
- GET /metrics
- Trace gRPC and HTTP calls.

Testing focus

- Health check test.
- Graceful shutdown test where practical.
- Metrics endpoint test.
- Trace propagation manual verification.

Exit criteria before moving to next phase

- You can see request latency in metrics.
- You can trace one order across services.
- Services shut down without losing requests.

Phase 15: DevOps, CI/CD, and Kubernetes

Item	Details
Difficulty	Expert
Suggested duration	2-3 weeks
Main goal	Package, deploy, and operate the project like a real backend system.

What you will build

- Dockerfiles for services.
- Docker Compose for local stack.
- GitHub Actions CI.

- Database migration pipeline.
- Kubernetes manifests or Helm chart.
- Ingress.
- Secrets/config maps.
- Horizontal scaling.

What you will learn in Go/backend

- Containerization.
- CI/CD.
- Kubernetes deployment.
- Service discovery.
- Config management.
- Secret management.
- Scaling.
- Rolling updates.

Database / schema work

- Run migrations as a job or init container.
- Separate local/dev/prod configs.
- Backup and restore strategy for PostgreSQL.

API / service work

- No new API required. Deploy all existing APIs.

Testing focus

- CI runs lint/test/build.
- Smoke test after deployment.
- Migration rollback test in staging.

Exit criteria before moving to next phase

- You can deploy the system from scratch.
- All services start from Docker Compose.
- CI catches broken builds.
- Kubernetes deployment is documented.

Part 4: Database Expansion Plan

Your existing schema is a catalog base. The project needs business tables around users, pharmacies, stock, orders, payments, prescriptions, notifications, and audit logs.

Area	Tables to add
Auth	users, roles, permissions, user_roles, role_permissions, refresh_tokens, password_resets
Catalog enhancement	medicine_prices, medicine_images, medicine_tags, medicine_search_logs, medicine_views
Pharmacy	pharmacies, pharmacy_branches, pharmacy_staff

Inventory	medicine_stocks, stock_batches, stock_movements, stock_reservations
Order	carts, cart_items, orders, order_items, order_status_logs, idempotency_keys
Prescription	prescriptions, prescription_items, prescription_reviews
Payment	payments, payment_transactions, refunds, invoices
Notification	notifications, notification_templates, notification_logs
Audit	audit_logs, login_logs, activity_logs
Events	outbox_events, consumed_events

Suggested naming cleanup

- Rename companie to companies when you start your own migrations.
- Rename companie_id to company_id.
- Rename packsizeandprice to pack_size_and_price, or split into pack_size and price if the data allows.
- Add created_at and updated_at to new tables from the start.
- Use UUID for business-facing IDs if you want stronger external API safety, but integer IDs are fine for learning.

Part 5: Important Service Flows

Order checkout flow

1. Customer adds medicine to cart.
2. API Gateway validates JWT and sends request to Order Service.
3. Order Service checks cart and calls Inventory Service to reserve stock.
4. Inventory Service locks stock rows and creates stock reservation.
5. Order Service creates order and order_items snapshot.
6. Order Service publishes OrderPlaced event.
7. Notification Service sends order confirmation.
8. Payment Service creates payment request if online payment is selected.

Payment callback flow

1. Payment gateway calls Payment Service webhook.
2. Payment Service verifies callback signature or token.
3. Payment Service checks idempotency to avoid duplicate processing.
4. Payment status changes to PAID or FAILED.
5. Payment Service publishes PaymentCompleted event.
6. Order Service updates order payment state.
7. Notification Service sends payment confirmation.

Prescription review flow

1. Customer uploads prescription.
2. Prescription Service stores file and creates PENDING review record.
3. Admin/pharmacist reviews prescription.
4. Prescription is approved or rejected.
5. Prescription Service publishes PrescriptionReviewed event.
6. Notification Service informs customer.
7. Approved prescription can be used for order confirmation.

Part 6: Suggested Repository Structure

```
oshudh-somadhan/  
  api-gateway/  
  services/  
    auth-service/  
    medicine-service/  
    inventory-service/  
    order-service/  
    payment-service/  
    prescription-service/  
    notification-service/  
    recommendation-service/  
  proto/  
    auth/  
    medicine/  
    inventory/  
    order/  
  shared/  
    logger/  
    config/  
    middleware/  
    errors/  
  migrations/  
  deployments/  
    docker-compose.yml  
    k8s/  
  docs/  
    architecture.md  
    api-design.md  
    database-design.md  
  scripts/  
  Makefile  
  README.md
```

Part 7: Suggested GitHub Milestones

Milestone	Issues you can create
-----------	-----------------------

Oshudh Somadhan - Go Backend Learning Roadmap

M1 - Foundation	Repo setup, Docker Compose, DB import, health endpoint, migration tool
M2 - Catalog API	Medicine list, search, detail, alternatives, pagination, filters
M3 - Architecture	Refactor handler/service/repository, error format, logger, config
M4 - Auth	Register, login, refresh, logout, RBAC, admin middleware
M5 - Search & Cache	Full-text search, Redis cache, autocomplete, rate limit
M6 - Admin	Admin CRUD, validation, audit logs
M7 - Inventory	Pharmacy, stock, batches, stock movement, low stock alert
M8 - Orders	Cart, checkout, order status, order history, idempotency
M9 - Prescription	Upload, review, approve/reject, attach to order
M10 - Payment	Payment initiation, webhook, invoice, refund
M11 - Events	RabbitMQ/NATS, notification worker, outbox pattern
M12 - gRPC	Proto files, internal services, interceptors
M13 - GraphQL	GraphQL schema, resolvers, dataloader
M14 - Quality	Unit tests, integration tests, race tests, benchmarks
M15 - Deploy	Dockerfiles, CI/CD, Kubernetes, monitoring

Final Starter Checklist

- Create GitHub repo: oshudh-somadhan.
- Create Docker Compose with PostgreSQL and Redis.
- Import backup.sql into local PostgreSQL.
- Create Go API with /health endpoint.
- Build medicine list API.
- Build medicine detail API with generic/company/dosage form join.
- Add search and alternatives by same generic.
- Write tests for each endpoint before adding auth.
- Only after the catalog works, start auth and RBAC.

Recommended rule: Do not start with microservices immediately. Build a modular monolith first, understand the domain, write tests, then split into services when boundaries become clear.